

TypeScript - Principiante

Tema 19: Manejo de errores

20 ejercicios progresivos, sin soluciones, para practicar throw, try/catch/finally, Error, errores personalizados, unknown en catch, validaciones y propagacion de errores en codigo sincrono y asincrono.

Ejercicio 1 - Lanzar un error

Crea una funcion divide que lance un error cuando el divisor sea cero.

Requisitos

- Usa throw.
- Lanza una instancia de Error.
- Devuelve number cuando la operacion sea valida.

Ejemplo de ejecucion

Resultado: 5

Error: no se puede dividir entre cero

Objetivo

Introducir throw.

Pista

Lanza el error cuando la funcion no pueda continuar correctamente.

Ejercicio 2 - Capturar un error

Ejecuta divide dentro de un bloque try/catch.

Requisitos

- Captura el error.
- Muestra un mensaje apropiado.
- No permitas que el programa termine inesperadamente.

Ejemplo de ejecucion

Error controlado

Objetivo

Practicar try/catch.

Pista

Coloca dentro de try solamente el codigo que puede fallar.

Ejercicio 3 - finally

Crea una operacion que pueda fallar y agrega un bloque finally.

Requisitos

- Muestra un mensaje desde finally.
- Prueba un caso exitoso y uno fallido.

Ejemplo de ejecucion

Operacion finalizada

Objetivo

Practicar finally.

Pista

finally se ejecuta independientemente de que exista error.

Ejercicio 4 - Validar edad

Crea una funcion registerUser que lance un error si la edad es menor al minimo permitido.

Requisitos

- Valida antes de crear el usuario.
- Captura el error desde el codigo consumidor.

Ejemplo de ejecucion

Error: edad no permitida

Objetivo

Usar errores para impedir operaciones invalidas.

Pista

No uses excepciones para condiciones normales si una validacion simple fuera suficiente; aqui practica deliberadamente throw.

Ejercicio 5 - Cantidad invalida

Crea una funcion calculateTotal que lance un error si precio o cantidad son negativos.

Requisitos

- Muestra mensajes claros.
- Devuelve el total cuando los datos sean validos.

Ejemplo de ejecucion

Total: \$1500

Objetivo

Practicar validaciones que lanzan errores.

Pista

Valida los argumentos al inicio de la funcion.

Ejercicio 6 - Error como unknown

En un catch, trata el error de forma segura.

Requisitos

- No uses any.
- Comprueba instanceof Error antes de acceder a message.
- Maneja tambien valores desconocidos.

Ejemplo de ejecucion

Error: dato invalido

Objetivo

Practicar manejo seguro del valor capturado.

Pista

No asumas que todo lo lanzado es necesariamente Error.

Ejercicio 7 - Error personalizado

Crea una clase `ValidationError` que herede de `Error`.

Requisitos

- Agrega una propiedad `field`.
- Usa `super`.
- Lanza el error al validar un usuario.

Ejemplo de ejecucion

Campo email: valor invalido

Objetivo

Introducir errores personalizados.

Pista

Una clase de error puede transportar contexto adicional.

Ejercicio 8 - `NotFoundError`

Crea `NotFoundError` para representar registros inexistentes.

Requisitos

- Incluye el nombre de la entidad y el id.
- Usalo al buscar un usuario.

Ejemplo de ejecucion

User 10 no encontrado

Objetivo

Modelar errores especificos.

Pista

Los tipos de error permiten distinguir diferentes causas.

Ejercicio 9 - Varios tipos de error

Crea ValidationError y NotFoundError y manejalos de forma diferente.

Requisitos

- Usa instanceof.
- Agrega un caso general para Error.

Ejemplo de ejecucion

Error de validacion
Registro no encontrado

Objetivo

Practicar narrowing de errores.

Pista

Ordena las comprobaciones desde errores especificos hacia casos generales.

Ejercicio 10 - Repositorio

Crea UserRepository y lanza NotFoundError cuando update o delete reciban un id inexistente.

Requisitos

- getByld puede devolver undefined.
- update y delete deben fallar explicitamente cuando corresponda.

Ejemplo de ejecucion

Usuario eliminado
Error: usuario no encontrado

Objetivo

Aplicar errores en una capa de datos.

Pista

Decide que operaciones pueden representar ausencia y cuales deben considerarla un error.

Ejercicio 11 - Duplicados

Agrega DuplicateEntityError y evita ids duplicados al insertar.

Requisitos

- Crea una clase de error propia.
- Incluye el id duplicado.
- Captura el error desde fuera del repositorio.

Ejemplo de ejecucion

Error: ya existe User con id 10

Objetivo

Practicar reglas de integridad.

Pista

El repositorio puede proteger la consistencia de su coleccion.

Ejercicio 12 - Servicio

Crea UserService que utilice UserRepository y maneje o propague errores segun corresponda.

Requisitos

- No ocultes errores sin motivo.
- Agrega contexto cuando sea util.
- Mantiene tipos claros.

Ejemplo de ejecucion

No fue posible actualizar el usuario

Objetivo

Comprender propagacion de errores entre capas.

Pista

No todos los errores deben resolverse en la misma capa donde aparecen.

Ejercicio 13 - Re-lanzar error

Captura un error, registra informacion y vuelve a lanzarlo.

Requisitos

- Usa throw nuevamente.
- El consumidor final tambien debe poder capturarlo.

Ejemplo de ejecucion

Error registrado

Operacion cancelada

Objetivo

Practicar rethrow.

Pista

Capturar no obliga a consumir definitivamente el error.

Ejercicio 14 - Preservar causa

Crea una operacion de servicio que falle debido a una operacion interna y genera un error de nivel superior conservando la causa cuando tu entorno lo permita.

Requisitos

- Agrega contexto al nuevo error.
- No pierdas innecesariamente la causa original.

Ejemplo de ejecucion

No se pudo completar la operacion

Objetivo

Practicar wrapping de errores.

Pista

Un error de alto nivel puede explicar que operacion fallo y conservar el error original.

Ejercicio 15 - Resultado vs excepcion

Implementa dos versiones de una busqueda: una devuelve undefined cuando no encuentra y otra lanza NotFoundError.

Requisitos

- Prueba ambas versiones.
- Escribe comentarios explicando cuando cada enfoque puede ser apropiado.

Ejemplo de ejecucion

Busqueda opcional: undefined

Busqueda requerida: error

Objetivo

Distinguir ausencia esperada de error excepcional.

Pista

No toda situacion negativa necesita una excepcion.

Ejercicio 16 - Error asincrono

Crea una funcion async que pueda rechazar y manejala con try/catch.

Requisitos

- Devuelve Promise.
- Simula un error de consulta.
- No uses any.

Ejemplo de ejecucion

Error al cargar usuario

Objetivo

Combinar errores con async/await.

Pista

Los rechazos de una Promise pueden manejarse con try/catch alrededor de await.

Ejercicio 17 - Varias operaciones async

Ejecuta varias operaciones asincronas dependientes y detiene el flujo cuando una falla.

Requisitos

- Usa try/catch.
- No ejecutes pasos posteriores si el anterior fallo.
- Muestra que paso produjo el problema.

Ejemplo de ejecucion

Usuario cargado

Error al cargar pedidos

Objetivo

Practicar propagacion en flujos asincronos.

Pista

Un solo try puede cubrir un flujo, pero conserva suficiente contexto para identificar el paso.

Ejercicio 18 - Errores de dominio

Crea InsufficientBalanceError para una cuenta bancaria.

Requisitos

- La cuenta no puede quedar negativa.
- Incluye saldo disponible y cantidad solicitada.
- Captura el error al intentar retirar.

Ejemplo de ejecucion

Saldo insuficiente: disponible \$500, solicitado \$800

Objetivo

Modelar errores de reglas de negocio.

Pista

El error puede contener datos utiles ademas del mensaje.

Ejercicio 19 - Mapeo de errores

Crea una funcion que convierta diferentes errores de dominio en un objeto `AppErrorResponse`.

Requisitos

- Maneja `ValidationError`, `NotFoundError` y errores desconocidos.
- La respuesta contiene `code` y `message`.
- No uses `any`.

Ejemplo de ejecucion

`NOT_FOUND` - Usuario no encontrado

Objetivo

Practicar transformacion de errores en respuestas controladas.

Pista

Centraliza la traduccion de tipos de error a codigos de aplicacion.

Ejercicio 20 - Sistema de gastos robusto

Construye `ExpenseRepository` y `ExpenseService` con manejo estructurado de errores.

Requisitos

- Crea `ValidationError`, `NotFoundError` y `DuplicateEntityError`.
- Valida concepto, categoria y cantidad.
- Evita ids duplicados.
- Lanza `NotFoundError` al actualizar o eliminar un gasto inexistente.
- Los metodos asincronos deben propagar errores correctamente.
- El codigo consumidor usa `try/catch`.
- Trata el error capturado como `unknown`.
- Distingue errores conocidos de errores inesperados.
- Usa `finally` en una operacion donde tenga sentido.
- No uses `any`.

Ejemplo de ejecucion

Gasto registrado

Error de validacion: cantidad invalida

Operacion finalizada

Objetivo

Integrar validacion, errores personalizados, repositorios, servicios y `async/await`.

Pista

Define primero que situaciones son errores esperados de dominio y despues decide en que capa se manejan.